Basic dense matrix multiplication in Python, Java and C: how matrix size affects runtime and memory

Big Data - Individual Assignment 1 (Basic Matrix Multiplication in Different Languages)

Hugo Amador Hidalgo - Grado en Ciencia e Ingeniería de Datos, Universidad de Las Palmas de Gran Canaria - academic year 2026-2027

Abstract. We measure the straightforward i-j-k triple-loop multiplication of two n × n float64 matrices in C (gcc -O2), Java (OpenJDK 21, JIT) and CPython 3.13, using identical input files, a monotonic timer around the multiplication kernel only, 3 to 10 repeated samples per configuration and the peak resident memory of the process. All runs come from one 2-vCPU virtual machine. Results of the three languages agree bit for bit, so the comparison is between equal computations. Runtime grows roughly with n^3 in every language, but the cost per multiply-add is not constant: it rises from 0.65 to 5.8 ns in C (n = 16 to 2048), from 0.96 to 12.3 ns in Java and stays between 44 and 109 ns in Python (n up to 1024). Python was 26 to 78 times slower than C and 24 to 54 times slower than Java. Java took between 0.8 and 2.2 times the time of C depending on n, with no monotonic pattern: about equal at n = 1024, about 2 times at n = 1536 and 2048 and 2.2 times at n = 1000. Peak memory follows the array storage in C, adds a fixed runtime of about 42 MiB in Java and is about 5 times the array storage in Python (boxed floats). Two supplementary experiments show that the cost per iteration also changes by up to a factor 2.3 between neighbouring sizes (1000, 1024, 1025), so every conclusion is scoped to the tested sizes, settings and hardware.

1. Introduction and objective

Matrix multiplication is a standard case study for performance engineering: its algorithm is simple, its arithmetic cost is known (n^3 multiply-adds for the textbook algorithm [3]) and its memory-access pattern exposes the memory hierarchy [4, 5]. The course material uses it to compare programming languages and to introduce benchmarking and performance-engineering rules [1, 10]. This study follows the common experimental protocol of the assignment [2].

Research question. How does the matrix size n affect the runtime and the memory use of the basic triple-loop multiplication implemented in Python, Java and C, when inputs, arithmetic, timing boundary and machine are controlled?

Hypotheses (stated before measuring; they are predictions, not results). H1: time per multiplication grows as n^3 in all languages. H2: C and Java are much faster than Python (C at least as fast as Java), and the differences are constant factors, not different growth orders. H3: peak memory of C is about the array storage $3 \times 8\ n^2$ bytes plus a small runtime; Java adds a larger fixed runtime; Python needs several times more because every element is a separate object. H4: once the matrices no longer fit in the caches the cost per multiply-add increases, so H1 holds only approximately.

2. Background and references

For C = AB with n × n matrices, C[i][j] = sum over k of A[i][k] B[k][j]. The triple loop performs n^3 multiply-add operations and touches $3n^2$ values [3]. In the i-j-k order the inner loop reads a row of A sequentially but walks down a column of B with a stride of n elements. Such strided accesses reuse cache lines poorly when a column no longer fits in the cache, which is why loop order and blocking matter [4, 5, 12]; here the loop order is fixed because the comparison is between languages, and optimisations belong to Assignment 2.

Floating-point addition is not associative. For non-negative terms the recursive summation of a dot product of length n has relative error at most $\gamma_n = nu/(1 - nu)$ with $u = 2^{-53}$ [6, 7]. This bound gives the correctness tolerance used below. Java has used strict IEEE-754 semantics for all code since JDK 17 [13] and the C code is compiled with -ffp-contract=off [14], so the three languages perform the same sequence of double-precision operations.

Benchmarking managed runtimes needs warm-up, repetition and robust summaries because JIT compilation and other effects change run-to-run times [8, 9]; this is why the protocol uses warm-up runs, repeated samples, the median and the interquartile range.

3. Methodology

3.1 Implementations

	C	Java	Python
Source	src/c/matmul.c	src/java/MatMul.java	src/python/matmul.py
Algorithm	i-j-k triple loop, local accumulator, no library call	same	same, pure Python (no NumPy)
Storage layout	one contiguous row-major double array per matrix	double[n][n]: array of contiguous row arrays	list of lists of Python floats; each element is a separate boxed object
Toolchain / settings	gcc 13.3.0 (Ubuntu 13.3.0-6ubuntu2~24.04.1); -O2 -std=c11 -ffp-contract=off	OpenJDK 21.0.12.1 2026-08-18; -XX:+UseSerialGC -Xmx3g, default tiered JIT	CPython 3.13.16 (no JIT, default settings)
Timer	clock_gettime(CLOCK_MONOTONIC)	System.nanoTime()	time.perf_counter_ns()

The unavoidable differences are the storage layouts (C: one flat array; Java: an array of row arrays with bounds checks; Python: boxed objects) and the execution model (native code, JIT, interpreter). The settings that are kept fixed are the loop order, the numeric type (float64), the accumulator pattern and the input files.

3.2 Inputs, correctness and tolerance

Matrices A and B are uniform in [0, 1), generated with NumPy's PCG64 generator [11] (seeds 20261012 + n and 20261013 + n) and stored as little-endian binary files, which all three programs read, so the logical inputs are identical (a shared file is preferable to language-specific random generators). A SHA-256 manifest (data/manifest.csv) allows regenerating and verifying them.

Before benchmarking, 53 checks passed (53 of 53): identity × M, M × identity, zero × M, M × zero, 1 × 1, 2 × 2 and 3 × 3 hand-computed products (exact equality, integer-valued data) and random matrices of sizes 2 to 128 compared over the full matrix with NumPy/BLAS as the trusted reference. For random inputs the relative tolerance is $2\gamma_n$ (atol = 0): both results satisfy the γ_n bound of Section 2, so they can differ by at most twice that. The three languages produced bitwise identical matrices in every test. For benchmark sizes each program prints a checksum (sum of all entries in row-major order) and three sample entries, validated against NumPy with tolerance $2\gamma_n$ for entries and $2(\gamma_n + \gamma_{n^2})$ for the checksum. All validated checksums were also bitwise identical across the languages and C optimisation levels (16 study/size groups compared, 0 mismatches).

3.3 Environment

Item	Value
CPU	Intel(R) Xeon(R) Processor @ 2.80GHz, 2 physical cores, 2 logical processors (virtual machine, hypervisor KVM); L1d 64 KiB (2 instances), L2 2 MiB (2 instances), L3 33 MiB (1 instance)
Memory / OS	7.8 GiB RAM; Linux-6.18.44-fc-v114-x86_64-with-glibc2.39
Threads	all kernels single-threaded; the JVM also runs background JIT-compiler threads; SerialGC avoids parallel GC workers; no library creates threads inside the timed region (NumPy is not used by the programs)
Frequency / affinity	CPU governor not exposed by the VM; no frequency control and no CPU pinning (limitation)
Run order	one configuration (language, n) at a time, languages interleaved for each n so that all three see similar machine conditions

3.4 Timing boundary, warm-up, repetitions

Timed region: only the multiplication kernel. Reading the inputs, allocating C, computing the checksum and writing output happen outside the timer. C is allocated and filled before the first multiplication; the loops write every element of C. Each program consumes the result (checksum printed and validated), so the computation cannot be removed.

Warm-up: C and Java run warm-up multiplications until at least 1 s has elapsed (at least one run) so that caches, page tables and the JIT-compiled kernel are in steady state; CPython runs the same warm-up for $n \leq 256$. For CPython at $n \geq 512$ no explicit warm-up was requested, because there is no JIT and one run takes more than 10 s. However the program executes one *untimed calibration multiplication* before sampling when no warm-up has run (it sizes the sample batch); it is effectively one warm-up run. The programs' own WARMUP counter shows 0 for these runs; summary.csv counts it as 1, which wall-clock accounting confirms (for example n = 768: 272 s of wall time versus 228 s of samples). The first timed sample differs by more than 10% from the median in C n = 16 (-16%), C n = 64 (+36%), C n = 256 (+14%), Java n = 16 (-21%), Java n = 32 (-11%), Java n = 256 (-14%), Java n = 768 (+14%), Java n = 1024 (+39%), Python n = 256 (+13%); no sample was removed.

Samples: 10 for n ≤ 256 and 5 for n ≥ 512, but at least 3 and stopping once 300 s of measured time is reached. Costly runs therefore have fewer samples: CPython n = 1024 (3 samples, 111 to 122 s) and Java n = 2048 (3 samples, 105 to 108 s). When one multiplication takes less than 0.1 s, a sample is a batch of repeated multiplications whose duration is about 0.1 s and the *time per multiplication* is stored (column inner_multiplications in raw_measurements.csv). Reported statistics: median and inter-quartile range (IQR); all raw samples are kept in results/raw_measurements.csv.

3.5 Memory metric, size sequence and budgets

Memory: peak resident set size (RSS) of the whole benchmark process, read from the kernel (ru_maxrss) through a small wrapper (scripts/rss_run.c); the wrapper is needed because a forked child inherits the parent's RSS high-water mark, which would add the harness's own ~30 MiB to every value. The metric includes the language runtime, input buffers and temporaries; it is not comparable to the ideal array size $3 \times 8\ n^2$ bytes without subtracting the runtime baseline, which was measured separately with n = 1 (C 1.6 MiB, Java 42.4 MiB, CPython 9.8 MiB; median of 5).

Sizes: n = 16, 32, 64, 128, 256, 512, 768, 1024, 1536, 2048 (powers of two plus 768 and 1536 as controls). **Budgets declared before the large runs:** one multiplication at most 180 s; one configuration at most 15 min of wall time; peak RSS at most 4 GiB. A configuration that breaks a budget is reported as such with no invented time, and larger sizes of that language are not attempted. The dense storage of three double matrices at n = 10 000 would need 2.4 GB, so n = 2048 (100 MiB) was never memory-limited.

4. Results

4.1 Runtime

n	C (-O2)	Java	CPython	samples C / Java / Python
16	2.68 us (IQR 14%)	3.94 us (IQR 4%)	209 us (IQR 9%)	10 / 10 / 10
32	21.6 us (IQR 25%)	32.7 us (IQR 22%)	1.44 ms (IQR 1%)	10 / 10 / 10
64	219 us (IQR 44%)	250 us (IQR 7%)	13.5 ms (IQR 12%)	10 / 10 / 10
128	1.9 ms (IQR 7%)	2.91 ms (IQR 5%)	112 ms (IQR 34%)	10 / 10 / 10
256	19.8 ms (IQR 12%)	23.1 ms (IQR 17%)	1.22 s (IQR 17%)	10 / 10 / 10
512	331 ms (IQR 8%)	258 ms (IQR 19%)	12.5 s (IQR 6%)	5 / 5 / 5
768	1.02 s (IQR 5%)	934 ms (IQR 17%)	46 s (IQR 9%)	5 / 5 / 5
1024	4.55 s (IQR 3%)	4.89 s (IQR 6%)	117 s (IQR 5%)	5 / 5 / 3
1536	18.6 s (IQR 2%)	37.2 s (IQR 2%)	timeout (15 min)	5 / 5 / 0
2048	50.1 s (IQR 7%)	106 s (IQR 1%)	not attempted	5 / 3 / 0

Table 1. Median time per multiplication (IQR as a percentage of the median). n ≤ 64 values are per-multiplication times obtained from batches. CPython at n = 1536 did not finish two multiplications within the 15 min wall limit (the single-run budget of 180 s is exceeded).

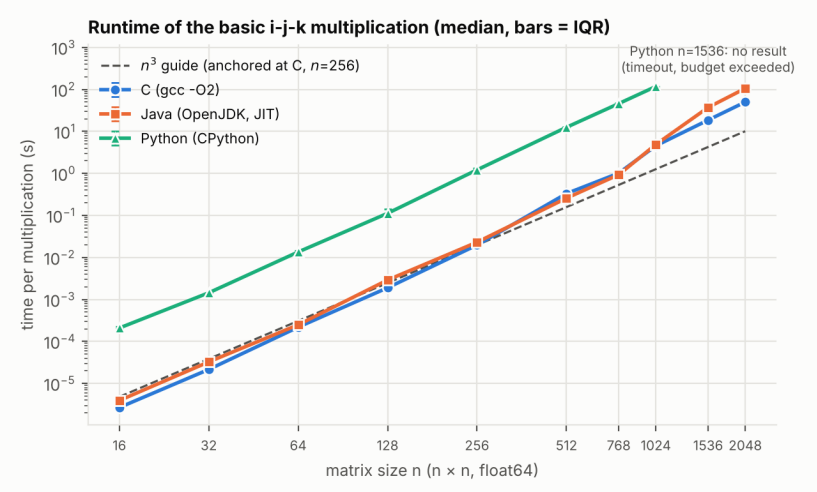


Figure 1. Time per multiplication versus n (log-log; median with IQR bars, which are smaller than the markers for most points). The dashed line is an n^3 guide anchored at the C median for n = 256.

n	C	Java	CPython	Java / C	CPython / C
16	0.65	0.96	51.1	1.47	78.1
32	0.66	1.00	44.1	1.51	66.8
64	0.84	0.95	51.6	1.14	61.6
128	0.91	1.39	53.6	1.53	59.2
256	1.18	1.38	72.5	1.17	61.6
512	2.47	1.92	93.3	0.78	37.8
768	2.24	2.06	101.4	0.92	45.3
1024	4.24	4.56	109.2	1.08	25.8
1536	5.14	10.27	-	2.00	-
2048	5.83	12.30	-	2.11	-

Table 2. Median time divided by n^3, in nanoseconds per inner-loop iteration (one multiply-add), and ratios of medians.

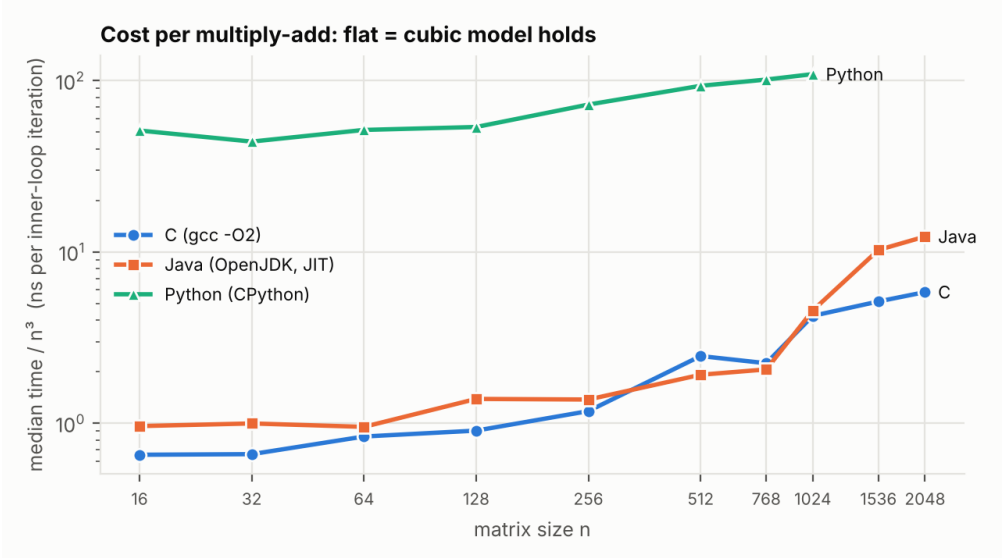


Figure 2. Cost per inner-loop iteration. A flat line would mean that the cubic model with a constant factor describes the data; none of the lines is flat.

A log-log least-squares fit of the medians for $n \geq 128$ gives exponents 3.71 (C, n up to 2048), 3.84 (Java, n up to 2048) and 3.34 (CPython, n up to 1024). These are descriptive slopes over a range in which the constant itself changes, not estimates of the algorithmic exponent: between consecutive sizes the local slope of the C medians varies from 2.8 to 5.2.

4.2 Memory

n	array storage $3 \times 8\ n^2$ (MiB)	C peak RSS (MiB)	Java peak RSS (MiB)	CPython peak RSS (MiB)
16	0.01	1.8	45.3	9.9
32	0.02	1.6	43.2	10.0
64	0.09	1.8	42.5	10.1
128	0.38	2.0	43.2	11.7
256	1.50	3.1	45.0	17.1
512	6.00	7.6	49.7	39.9
768	13.50	15.1	57.9	77.5
1024	24.00	25.5	91.0	130.3
1536	54.00	55.5	119.2	-
2048	96.00	97.5	145.6	-
baseline (n = 1)	-	1.6	42.4	9.8

Table 3. Peak resident set size of the benchmark process, including the language runtime.

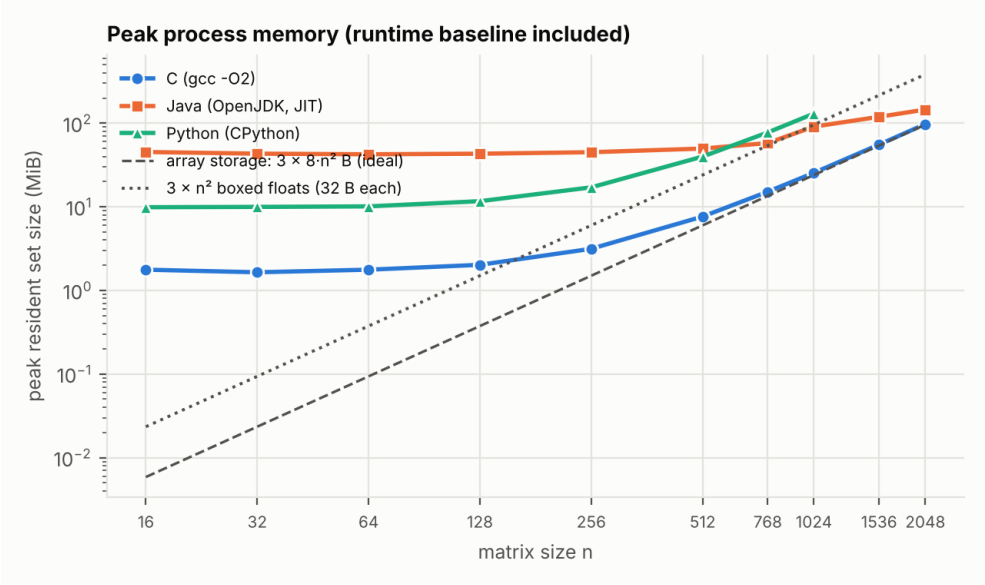


Figure 3. Peak RSS. The dashed line is the ideal storage of three float64 matrices; the dotted line assumes three matrices of boxed Python floats at 32 bytes per element (24 bytes object + 8 bytes pointer).

After subtracting the baseline, C at n = 1024 uses 23.9 MiB, essentially the 24.0 MiB of array storage. Java uses 48.7 MiB at n = 1024 (2.0 times the arrays) but 103 MiB at n = 2048 (1.08 times the 96 MiB of arrays). CPython uses 121 MiB at n = 1024, 5.0 times the array storage and a little above the 96 MiB expected from boxed floats; the excess is consistent with temporary buffers while the inputs are loaded, which was not measured separately.

4.3 Limits

Language	Largest tested size within the budgets	Largest size attempted	Remark
C	2048	2048	single multiplication 50 s at n = 2048; budgets not binding
Java	2048	2048	single multiplication 106 s at n = 2048; 3 samples because of the 300 s sample budget
Python	1024	1536	117 s per multiplication at n = 1024 (3 samples); n = 1536 timed out; n = 2048 not attempted

Table 4. Largest tested size under the declared budgets versus largest size attempted. Memory (maximum 146 MiB) never limited a run.

4.4 Supplementary experiments (kept apart from the controlled comparison)

S1 - C optimisation level. The same C source compiled with -O0, -O2 and -O3 (n = 256, 512, 1024; same inputs, protocol and checksums). **S2 - neighbouring sizes.** C (-O2) and Java at n = 1000, 1024 and 1025, run back to back with the same protocol.

n	-O0	-O2	-O3
256	49 ms (IQR 19%)	19.9 ms (IQR 12%)	19.2 ms (IQR 10%)
512	722 ms (IQR 9%)	397 ms (IQR 25%)	330 ms (IQR 30%)
1024	9.68 s (IQR 6%)	3.8 s (IQR 5%)	4.49 s (IQR 8%)

Table 5. Supplementary S1: median time per multiplication of the C kernel at three optimisation levels.

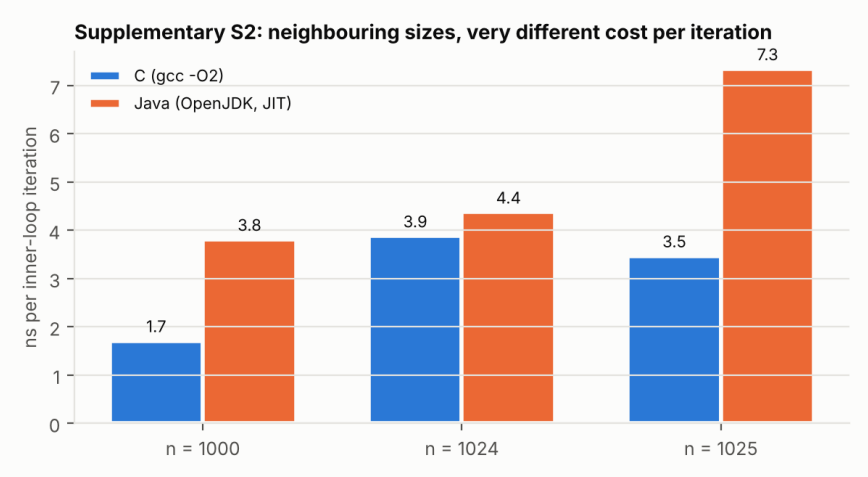


Figure 4. Supplementary S2: median cost per inner-loop iteration at three neighbouring sizes (C 1.7, 3.9, 3.5 ns; Java 3.8, 4.4, 7.3 ns; IQR up to 35% of the median).

5. Discussion

Answer to the research question. In all three languages the runtime grows approximately as n^3: for $n \leq 256$ the local slopes between consecutive sizes are 2.8 to 3.5 (Table 1). The cost per multiply-add, however, does not stay constant (Figure 2), so H1 holds as a first-order model but the constant factor depends on n. For C it rises from 0.65 ns at n = 16 to 1.18 ns at n = 256 and 5.8 ns at n = 2048; for Java from 0.96 to 1.38 and 12.3 ns. H4 is supported in direction: with 1 MiB of L2 per core and a 33 MiB shared L3 (Section 3.3), matrix B alone ($8n^2$ bytes: 2 MiB at n = 512, 8 MiB at n = 1024, 32 MiB at n = 2048) leaves the private cache between n = 362 and 512 and approaches the L3 size at n = 2048, while the column-wise walk over B reuses cache lines poorly. We did not use hardware counters (not exposed in this VM), so this is an explanation consistent with the data, not a measured cause.

Language effect. CPython is slower than C by a factor of 26 to 78 and than Java by 24 to 54, with a nearly constant cost of about 44 to 54 ns per iteration for $n \leq 128$ (roughly 123 to 150 clock cycles at 2.8 GHz), which indicates that the interpreter's per-iteration work, not memory, dominates for small n. The ratio to C shrinks for large n because the cost per iteration of C grows more (0.65 to 4.24 ns between n = 16 and 1024, a factor 6.5) than that of CPython (51 to 109 ns, a factor 2.1); this is consistent with C becoming limited by memory access, which we did not measure directly. We did not profile the interpreter, so we do not attribute the cost to individual bytecodes.

C versus Java. The ratio Java/C of the medians is 0.78 to 2.23 over all 13 pairs of measurements (the 10 main sizes plus the three S2 sizes; n = 1024 was measured in both) and has no monotonic pattern: 0.78 at n = 512, 1.08 at 1024, 2.00 at 1536, 2.11 at 2048, but 2.23 at n = 1000 and 2.12 at n = 1025. The identical C configuration measured in different processes varied as well: n = 512: 331 ms to 397 ms; n = 1024: 3.8 s to 4.55 s (factors 1.20 and 1.20). Differences below about 20% between C and Java for $n \leq 1024$ are therefore not distinguishable from this between-process variation, and H2's ordering (C at least as fast as Java) is supported only for the sizes where Java is clearly slower. A plausible reason for the slow Java cases is the double[][] layout: each access to B[k][j] first loads the row reference B[k] and rows are separate heap objects, whereas C uses one flat array. This was not isolated (no flat-array Java variant was run), so it remains a hypothesis. A comparison at a single size would give different rankings than one at another size, so statements such as “C is faster than Java” are not supported in general by these data.

Supplementary S1. Optimisation matters as much as the language: at n = 1024 the unoptimised C build (-O0) is 2.5 times slower than -O2, whereas -O3 is not consistently different from -O2 (n = 512: 330 ms versus 397 ms; n = 1024: 4.49 s versus 3.8 s; IQRs of 5% to 30%). This fixed-flag result is why the main study documents -O2 explicitly.

Supplementary S2 and fragility of the cost model. Neighbouring sizes differ strongly: in C the cost per iteration is 1.7 ns at n = 1000 but 3.9 and 3.5 ns at n = 1024 and 1025; in Java 3.8, 4.4 and 7.3 ns. The slow case is not only the power of two (n = 1025 is also slow), and the Java/C ranking changes with n. We therefore cannot name a mechanism from time measurements alone; row length in bytes (alignment and set mapping), TLB reach and hardware prefetching are candidates that would need counters to test. The practical consequence for the

main comparison is that every figure in Tables 1 and 2 is specific to its n.

Memory. H3 is supported: C needs the array storage plus about 1.6 MiB; Java adds roughly 42 MiB of fixed runtime and, at some sizes, extra memory (2.0 times the arrays at n = 1024 but 1.08 times at n = 2048; we did not investigate the cause, JVM heap sizing and garbage collection are candidates); CPython needs about 5 times the array storage because each float is a separate object. Peak RSS mixes arrays, runtime and temporaries, so these figures are comparable only as process footprints.

Limitations. (i) One virtual machine with unknown neighbours and uncontrolled CPU frequency; absolute times will differ on other hardware, and the ratios may too. (ii) Each configuration is one process; the IQR covers only variation within one process, and the same C configuration differed between processes by up to a factor 1.20 (Section 5). (iii) One pair of input matrices per size. (iv) Fewer than five samples for the two costliest runs, which gives a wide uncertainty for their medians. (v) The memory metric includes the runtime and was not split into managed heap versus native memory. (vi) The numbers on the course website (for example C 0.68 s versus Java 7.76 s for n = 1024 [1]) were obtained on an unspecified machine with unspecified compiler flags; we did not reproduce the 11-fold C advantage: here C takes 4.55 s and Java 4.89 s at n = 1024. Differences in loop order, compile-time array sizes or flags could explain it but we have not tested them.

6. Conclusions

Within the tested conditions (one 2-vCPU VM; n = 16 to 2048; float64; i-j-k loops; C at -O2, OpenJDK 21 with default JIT, CPython 3.13) the study establishes that: (1) the three implementations compute bitwise identical results, so the timing comparison is between equal computations; (2) runtime follows the cubic model only to first order, because the cost per multiply-add grows with n (consistent with B leaving the private cache, which was not measured directly); (3) CPython is 24 to 78 times slower than the compiled languages across the tested range (n up to 1024), a difference of constant factor, not of growth order; (4) Java is never clearly faster than C here and takes 0.8 to 2.2 times as long depending on n (about twice as long at n = 1000, 1025, 1536 and 2048, equal at 1024), so no single ranking holds at all sizes; (5) memory follows the array storage in C, a fixed runtime plus the arrays in Java and about five times the arrays in CPython. What remains unresolved: the mechanism behind the 2× changes between neighbouring sizes (S2), whether the Java gap at large n is due to the array layout, and how the results change on other hardware. Assignment 2 (optimised methods) is the natural place to test layout and loop-order explanations.

7. Use of generative AI

In line with the course policy, the following disclosure applies: the AI assistant Claude (Anthropic, model Sonnet 5.5, claude.ai) was used to plan the experiment, to draft the three implementations, the harness and analysis scripts, the README and the first draft of this report. All numerical results come from measurements stored in results/ and produced by the committed code; the AI output is not used as evidence. The author remains responsible for understanding the code, validating the measurements, verifying the references and defending the conclusions.

References

[1] M. D. Afonso Suárez, J. J. Hernández Cabrera and J. Évora Gómez, “Big Data,” course materials (Complexity Management; Monitoring and Performance Engineering), Universidad de Las Palmas de Gran Canaria, 2026/27. [Online]. Available: https://mltechdrawer.github.io/BIGDATA/

[2] “Individual Assignments: Matrix Multiplication,” Big Data, Grado en Ciencia e Ingeniería de Datos, ULPGC, revised proposal 2026-2027.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[4] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[5] U. Drepper, “What every programmer should know about memory,” Red Hat, Inc., Nov. 2007.

[6] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA, USA: SIAM, 2002.

[7] D. Goldberg, “What every computer scientist should know about floating-point arithmetic,” *ACM Computing Surveys*, vol. 23, no. 1, pp. 5-48, 1991.

[8] A. Georges, D. Buytaert and L. Eeckhout, “Statistically rigorous Java performance evaluation,” in *Proc. OOPSLA*, 2007, pp. 57-76.

[9] T. Kalibera and R. Jones, “Rigorous benchmarking in reasonable time,” in *Proc. ISMM*, 2013, pp. 63-74.

[10] J. L. Bentley, *Writing Efficient Programs*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1982.

[11] C. R. Harris *et al.*, “Array programming with NumPy,” *Nature*, vol. 585, pp. 357-362, 2020.

[12] M. S. Lam, E. E. Rothberg and M. E. Wolf, “The cache performance and optimizations of blocked algorithms,” in *Proc. ASPLOS IV*, 1991, pp. 63-74.

[13] “JEP 306: Restore always-strict floating-point semantics,” OpenJDK. [Online]. Available: https://openjdk.org/jeps/306

[14] GCC Manual, “Options that control optimization.” [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Optimize-Options.html

Reproducibility materials

Repository: https://github.com/hugoamadoor/BIG-DATA. It contains the three implementations, build.sh, a README with exact build and run commands (requirements.txt pins the Python packages), the input generator with its SHA-256 manifest, the test, benchmark and analysis scripts, the raw measurements (301 samples in results/raw_measurements.csv, one row per process in results/runs.csv), the environment record (results/environment.json), the summaries and all figures. The binary inputs (about 130 MiB) are regenerated with scripts/gen_inputs.py. Suggested archive: Assignment01_AmadorHidalgo_Materials.zip.